OPENTEXTSUMMIT

TECHNICAL HANDOVER DOCUMENT

OpenTextSummit EU Summit Website

Complete setup and maintenance guide for KOMBIT A/S

PREPARED BY

QXMBG — Monty Bagati

PREPARED FOR

KOMBIT A/S

DELIVERED AS

ZIP archive of source code

DATE

May 2026

Table of Contents

01 What You Received

02 ⚠ Before Anything Else — Replace These Two Values

03 File Reference

04 How the Site Works

05 How the Build Pipeline Works

06 Setup — Step by Step

07 Credentials Reference

08 Go-Live Checklist

09 Local Development

10 Maintenance Reference

SECTION 01

What You Received

OpenTextSummit is a **static website** built for the EU Summit conference. It is fully login-gated — visitors must register or sign in before accessing any content.

Property	Detail
Type	Static HTML — no server, no backend runtime
Hosting	GitHub Pages, deployed automatically via GitHub Actions
Authentication	Supabase Auth — email/password, Google, GitHub, magic link
Database	None — Supabase is used for authentication only
Analytics	Umami (cookieless, GDPR-friendly)
CSS Framework	Tailwind CSS v3.4.17
Auth SDK	@supabase/supabase-js v2.105.1

Pages

File	Accessible to	Purpose
`login.html`	Everyone	Sign-in, sign-up, password recovery, OAuth
`index.html`	Signed-in users only	Main landing page
`methodology.html`	Signed-in users only	QA cost estimation methodology
`privacy.html`	Everyone	GDPR privacy policy
`terms.html`	Everyone	Terms of service

SECTION 02

⚠ Before Anything Else — Replace These Two Values

Action required before you deploy.

The ZIP contains two values hardcoded to the original developer's personal analytics account. If you deploy without replacing them, your site's visitor data will be sent to the wrong account.

Values to Replace

File	Line	Find this string	Replace with
`index.html`	5	`data-website-id="4146a71b-3238-4a74-be59-c9a8d300212b"`	Your own Umami website ID
`index.html`	1433	`src="https://cloud.umami.is/p/069iZhcEV"`	Your own Umami pixel URL

Option A — Get Your Own Umami Account (Free)

1. Sign up at **cloud.umami.is**
2. Add a new website — enter your GitHub Pages URL as the domain
3. Copy the `data-website-id` value and the pixel `src` from your tracking snippet
4. Replace both values in `index.html`

Option B — Remove Analytics Entirely

Delete these two blocks from `index.html`:

```
<!— Delete lines 3-6: the Umami script tag —>
<script defer src="https://cloud.umami.is/script.js"
```

```
        data-website-id="4146a71b-3238-4a74-be59-c9a8d300212b"
        crossorigin="anonymous"></script>

<!-- Delete line 1433: the tracking pixel -->
<img src="https://cloud.umami.is/p/O69iZhcEV" class="tracking-pixel" alt="" />
```

What You Do NOT Need to Change

The following strings appear in the source files but are **not** hardcoded credentials — the build pipeline replaces them automatically using GitHub Actions secrets:

```
process.env.SUPABASE_URL   ← in index.html, login.html, methodology.html
process.env.SUPABASE_KEY   ← in index.html, login.html, methodology.html
```

> **Do not edit these manually.** They will be substituted correctly during the first deployment.

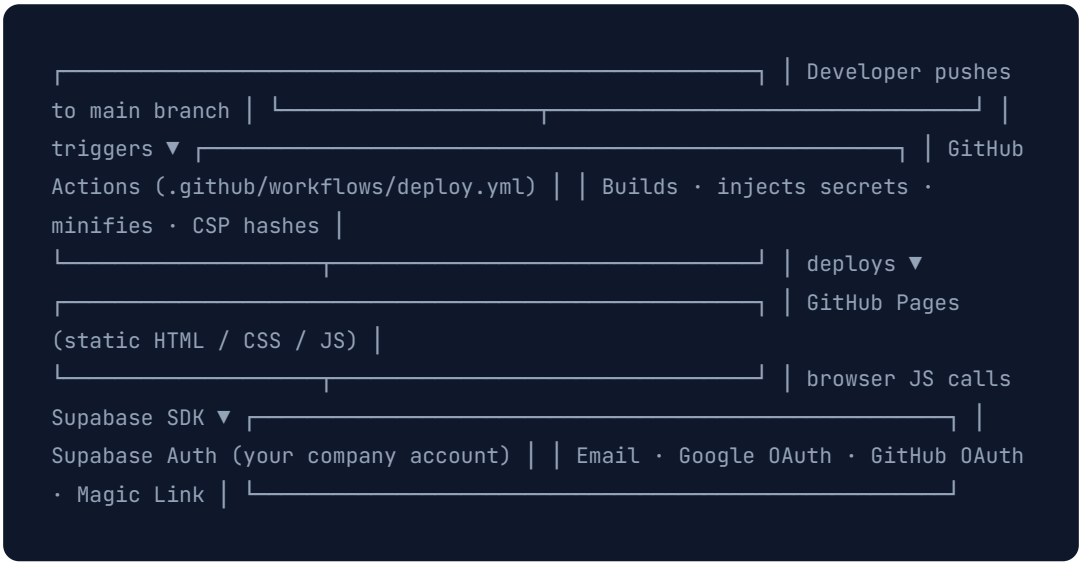
SECTION 03

File Reference

File / Path	Purpose
`index.html`	Main landing page — requires login
`login.html`	Sign-in / sign-up / OAuth page — public
`methodology.html`	QA cost estimation methodology — requires login
`privacy.html`	GDPR privacy policy — public
`terms.html`	Terms of service — public
`tailwind.config.js`	Tailwind CSS theme with brand colours
`tailwind.css`	Generated stylesheet — built by CI, do not edit manually
`_headers`	Per-page HTTP security headers (CSP, HSTS, X-Frame-Options)
`scripts/inject-csp-hashes.js`	Computes SHA-256 hashes of all inline scripts/styles and writes them into `_headers` — runs after minification
`.github/workflows/deploy.yml`	Full CI/CD pipeline — triggers on every push to `main`

SECTION 04

How the Site Works



Key Design Facts

- **No server.** Everything runs in the browser using the Supabase JavaScript SDK loaded from a CDN.
- **No database.** Supabase is used for authentication only — there are no custom tables or stored data beyond user accounts.
- **Credentials are never committed.** Source files contain placeholder strings. Real values are stored as GitHub Actions secrets and injected at build time.
- **Security policy is auto-computed.** A Content Security Policy with SHA-256 hashes for every inline script and style block is computed fresh on every deployment.
- **Inactivity timeout.** Authenticated pages sign users out after 5 minutes of inactivity, with a 1-minute warning banner.

SECTION 05

How the Build Pipeline Works

Defined in `.github/workflows/deploy.yml`. Triggers automatically on every push to `main`.

#	Step	What it does
1	**Checkout**	Fetches source code
2	**Build Tailwind CSS**	Generates `tailwind.css` from all HTML files
3	**Inject secrets**	Replaces `process.env.SUPABASE_URL` and `process.env.SUPABASE_KEY` with real values from GitHub Actions secrets
4	**Narrow CSP**	Replaces the wildcard `*.supabase.co` with your exact Supabase project hostname
5	**Verify secrets**	Fails the build immediately if any placeholder remains
6	**Minify HTML**	Minifies all 5 HTML files including inline JS and CSS
7	**Compute CSP hashes**	SHA-256 hashes every inline `<script>` and `<style>` block and writes hashes into `_headers`
8	**Deploy to Pages**	Publishes to GitHub Pages

> **Why order matters:** Hashes (step 7) must be computed after minification (step 6), because minification changes the content of inline scripts and styles.

SECTION 06

Setup — Step by Step

1 Create a GitHub Repository

1. In your company's GitHub organisation, create a new repository named `OpenTextSummit`.
2. Clone it locally:

```
git clone https://github.com/<your-org>/OpenTextSummit.git
cd OpenTextSummit
```

1. Extract the ZIP contents into this folder. All files must sit at the root — not inside a subfolder.
2. Commit and push:

```
git add .
git commit -m "chore: initial commit from ZIP handover"
git push origin main
```

2 Enable GitHub Pages

1. Go to your repository on GitHub.
2. Navigate to **Settings → Pages**.
3. Under **Source**, select **GitHub Actions**.
4. Click **Save**.

Your site will be at: `https://<your-org>.github.io/OpenTextSummit/`

> **Note this URL** — you will need it in Steps 4, 5, and 6.

3 Create a Supabase Project

1. Sign up at **supabase.com** — the free tier is sufficient.
2. Click **New project**, enter a name and database password, wait ~2 minutes for provisioning.
3. Go to **Project Settings → API** and copy:
   - **Project URL** → this becomes `SUPABASE_URL`
   - **anon public** key → this becomes `SUPABASE_KEY`

> The anon key is designed for browser JavaScript. It is not a password — it only allows public access and is safe to expose. **Never use the `service_role` key here.**

4 Configure Supabase Auth

1. Go to **Authentication → Providers** and enable **Email**.
2. Go to **Authentication → URL Configuration**:
   - Set **Site URL** to your GitHub Pages URL from Step 2
   - Under **Redirect URLs**, add your GitHub Pages URL

5 Set Up Google OAuth

IN SUPABASE DASHBOARD

1. Go to **Authentication → Providers → Google**.

2. Note the **Callback URL**: `https://<project-id>.supabase.co/auth/v1/callback`

IN GOOGLE CLOUD CONSOLE (CONSOLE.CLOUD.GOOGLE.COM)

1. Create or select a project.
2. Go to **APIs & Services → OAuth consent screen** and configure it (External).
3. Go to **Credentials → Create Credentials → OAuth 2.0 Client ID**.
4. Application type: **Web application**.
5. Under **Authorised redirect URIs**, add the Supabase Callback URL.
6. Copy the **Client ID** and **Client Secret**.

BACK IN SUPABASE

1. Paste the Client ID and Client Secret into the Google provider settings.
2. Toggle Google **on** and save.

6 Set Up GitHub OAuth

IN SUPABASE DASHBOARD

1. Go to **Authentication → Providers → GitHub**.
2. Note the **Callback URL** (same format as Google).

IN GITHUB

1. Go to **Settings → Developer settings → OAuth Apps → New OAuth App**.
2. Fill in:
   - **Application name:** OpenTextSummit
   - **Homepage URL:** your GitHub Pages URL
   - **Authorization callback URL:** the Supabase Callback URL
3. Click **Register application** then copy the **Client ID** and generate a **Client Secret**.

BACK IN SUPABASE

1. Paste the Client ID and Client Secret into the GitHub provider settings.
2. Toggle GitHub **on** and save.

7 Add GitHub Actions Secrets

1. In your repository, go to **Settings → Secrets and variables → Actions**.
2. Click **New repository secret** and add both:

Secret name	Value
`SUPABASE_URL`	Project URL from Supabase (e.g. `https://abcdefghij.supabase.co`)
`SUPABASE_KEY`	The **anon public** key from Supabase

> These are the only two secrets required. The build will fail with a clear error if either is missing.

8 Trigger First Deployment

1. Push any commit to `main`, or go to the **Actions** tab and click **Run workflow**.
2. Watch the workflow run — all 8 steps should go green.
3. If a step fails, the error message will state exactly which secret or configuration is missing.
4. Once complete, open your GitHub Pages URL to confirm the site is live.

SECTION 07

Credentials Reference

Name	Where to find it	Used for
`SUPABASE_URL`	Supabase → Project Settings → API → **Project URL**	Injected into HTML at build time; narrows CSP to exact Supabase host
`SUPABASE_KEY`	Supabase → Project Settings → API → **anon public**	Injected into HTML at build time; public browser-safe key
Google OAuth Client ID	Google Cloud Console → Credentials	Enables "Sign in with Google"
Google OAuth Client Secret	Google Cloud Console → Credentials	Stored in Supabase only, never in GitHub
GitHub OAuth Client ID	GitHub → Developer settings → OAuth Apps	Enables "Sign in with GitHub"
GitHub OAuth Client Secret	GitHub → Developer settings → OAuth Apps	Stored in Supabase only, never in GitHub

⚠ **Critical:** Supabase also provides a `service_role` key. This key bypasses all security rules and must **never** be used in this application or stored in GitHub. Always use the **anon public** key only.

SECTION 08

Go-Live Checklist

Work through this after completing all 8 setup steps.

- [ ] GitHub Actions workflow run shows all steps green
- [ ] GitHub Pages URL opens and the landing page loads correctly
- [ ] `login.html` renders the sign-in form without errors
- [ ] Email/password sign-up creates an account successfully
- [ ] Email/password sign-in works
- [ ] **Sign in with Google** completes the full OAuth flow and redirects back to the site
- [ ] **Sign in with GitHub** completes the full OAuth flow and redirects back to the site
- [ ] Visiting `index.html` while signed out redirects to `login.html`
- [ ] Visiting `methodology.html` while signed out redirects to `login.html`
- [ ] Both pages load correctly after signing in
- [ ] Leaving the site idle for 5 minutes shows a warning banner, then signs out automatically
- [ ] Browser DevTools → Console shows no Content-Security-Policy violation errors

SECTION 09

Local Development

For developers who want to run the site locally without deploying to GitHub Pages.

Requirements: Node.js 20+

```
# 1. Clone your company's repository
git clone https://github.com/<your-org>/OpenTextSummit.git
cd OpenTextSummit

# 2. Generate the CSS
npx tailwindcss@3.4.17 -c tailwind.config.js --content "*.html" -o tailwind.css

# 3. Substitute Supabase credentials locally (do NOT commit these changes)
sed -i "s|process.env.SUPABASE_URL|https://your-project.supabase.co|g" \
    index.html login.html methodology.html
sed -i "s|process.env.SUPABASE_KEY|your-anon-key-here|g" \
    index.html login.html methodology.html

# 4. Serve locally
npx serve .
# Site is now at http://localhost:3000
```

Before committing any changes

, restore the placeholder strings so credentials are not accidentally committed:

```
git checkout index.html login.html methodology.html
```

OAuth on localhost: Google and GitHub OAuth will not work on `localhost` unless you add `http://localhost:3000` to Supabase Redirect URLs and to each OAuth app's allowed callback URLs. Use email/password login for local development.

SECTION 10

Maintenance Reference

Updating the Supabase SDK

The SDK is loaded from a CDN with a pinned version and integrity hash in `index.html`, `login.html`, and `methodology.html`. To upgrade, update the version number **and** the `integrity` hash in all three files. Compute the new hash with:

```
curl -s https://cdn.jsdelivr.net/npm/@supabase/supabase-js@<version>/dist/umd/supabase.js | \
  openssl dgst -sha384 -binary | openssl base64 -A | sed 's/^/sha384-/'
```

Updating Tailwind CSS

The version is pinned in `.github/workflows/deploy.yml` as `tailwindcss@3.4.17`. To upgrade, update that version string. **Do not upgrade to Tailwind v4 without testing** — it has a completely different configuration format.

CSP Hashes — No Action Needed

The Content Security Policy hashes in `_headers` are **recomputed automatically on every deployment** by `scripts/inject-csp-hashes.js`. You never manage them manually.

Adding a New Page

If you add a new HTML page with inline scripts or authentication:

1. Add the page filename to the minification step in `.github/workflows/deploy.yml`
2. Add the page filename to the `HTML_FILES` array at the top of `scripts/inject-csp-hashes.js`
3. Add the appropriate security header entries for the new page to `_headers`

GitHub Actions Version Pins

All `uses:` entries in `deploy.yml` are pinned to full commit SHAs — not floating version tags. This is a supply-chain security measure. To update an action, find the SHA for the desired release in that action's GitHub repository and update the reference in `deploy.yml`.

End of document — OpenTextSummit Technical Handover — KOMBIT A/S — May 2026